Ingeniería Informática
ALFA

Práctica 4: Expresiones Regulares en LINUX

Ejercicio 0:

Operador Teórico	Operador en egrep	Descripción
Concatenación	No tiene operador explícito	Une símbolos sin separación, ej. abc representa a seguido de b seguido de c
Unión (+)	`	` (pipe)
Cierre de Kleene (*)	*	Representa 0 o más repeticiones, ej. a* reconoce ε, a, aa, aaa...
Cierre positivo (+)	+	Representa 1 o más repeticiones, ej. a+ reconoce a, aa, aaa...
Opcionalidad (ε)	?	Representa 0 o 1 aparición, ej. a? reconoce ε y a
Rangos	[ ]	Define conjuntos de caracteres, ej. [abc] equivale a a + b + c
Negación	[^]	Representa cualquier símbolo excepto los indicados, ej. [^0-9] reconoce todo menos dígitos
Inicio de línea	^	Reconoce el comienzo de línea, ej. ^ab reconoce líneas que empiezan con ab
Fin de línea	$	Reconoce el final de línea, ej. z$ reconoce líneas terminadas en z
Subexpresiones	()	Agrupa expresiones para aplicar operadores, ej. (abc)*
Número de repeticiones	{n,m}	Indica repeticiones mínimas y máximas, ej. a{2,4} reconoce aa, aaa, aaaa
Palabras completas	\< \> o \b	Delimita palabras completas, ej. \<dos\> reconoce dos, pero no adosado
Comodín	.	Representa cualquier carácter excepto salto de línea, ej. a.b reconoce a0b, a1b, a2b

Ejercicio 1:
El comando *egrep '2'* busca la aparición del símbolo 2 en cualquier parte de las líneas del archivo y lo colorea en azul o rojo. En 22 y 222, hay al menos un 2, por lo que se reconocen. Si interpretamos la salida como el reconocimiento de 22*, significa que egrep encuentra el 2 y luego cualquier cantidad adicional de 2, incluso 0 veces.

Ejercicio 2:

a) 11 + (0+1+2)0
egrep '11|[012]0' fichero.txt

b) b*c + aaa*b
egrep 'b*c|aaa*b' fichero.txt

c) Σ* para Σ = {a, b, c}
egrep '^[abc]*$' fichero.txt

d) (ε + a + b)bb(aa)*
egrep '(^|a|b)bb(aa)*' fichero.txt

e) ab + (ba)*c, al final de línea
egrep '(ab|(ba)*c)$' fichero.txt

Ejercicio 3:

egrep '(1|2|3)*|[^a-z]' fichero.txt
- (1|2|3)* reconoce cualquier secuencia compuesta por 1, 2 o 3, incluso la cadena vacía.
- [^a-z] reconoce cualquier carácter que no sea una letra minúscula.
Cómo está unido con |, reconoce cualquier línea que contenga solo 1, 2 o 3, o cualquier línea que contenga al menos un carácter que no sea una letra minúscula de la a-z.
Por ejemplo: “112333AZX”

egrep '^00|[^0]*' bits.txt
- ^00 reconoce líneas que comienzan con 00.
- [^0]* reconoce cualquier línea que no contenga el carácter 0 en ningún lugar.
Por ejemplo: “00111”

egrep '1*|0*' bits.txt
- 1* reconoce cualquier cantidad (incluso cero) de 1.
- 0* reconoce cualquier cantidad (incluso cero) de 0.
- Como está separado por |, la expresión acepta líneas con solo 1, solo 0, combinaciones de ambos, o incluso la cadena vacía.
Por ejemplo: “11000”, “0000”

Ejercicio 4:

a) Prueba la expresión '[^[:digit:]]*' con una cadena sin dígitos y explica su comportamiento. ¿Es equivalente a '[[:alpha:]]*'?.

[^[:digit:]]* → encuentra cualquier secuencia de caracteres que no contenga dígitos.
'[[:alpha:]]*' → solo coincide con secuencias de letras (A-Z, a-z).

Por tanto, la expresión '[^[:digit:]]*' no es equivalente a '[[:alpha:]]*', ya que '[^[:digit:]]*' también puede incluir espacios y signos de puntuación, mientras que '[[:alpha:]]*' solo coincide con secuencias de letras (A-Z, a-z).

```
paula@ubuntu:~$ echo "Hola, como estas?" | egrep '[^[:digit:]]*'
paula@ubuntu:stas?
paula@ubuntu:~$ echo "Hola, como estas?" | egrep '[[:alpha:]]*'
Hola, como estas?
```

En rojo se muestran los caracteres reconocidos, y en blanco los que no.

b) Indicar qué cadenas reconoce la siguiente instrucción: egrep '[^aeiouAEIOU]*' fichero.txt.

[^aeiouAEIOU]* → Encuentra secuencias de caracteres que no contienen vocales (mayúsculas ni minúsculas). Puede coincidir con cualquier conjunto de consonantes, espacios, números o signos de puntuación.

```
paula@ubuntu:~$ echo "Hola, como estas?" | egrep '[[^aeiouAEIOU]]*'
Hola, como estas?
```

c) Las matrículas de un coche están formadas por cadenas de 4 dígitos seguidos de 3 letras. Escribir la instrucción usando los alfabetos predefinidos. Considera la matrícula como una palabra.

egrep '\b[[:digit:]]{4}[[:upper:]]{3}\b' fichero.txt

\b → Asegura que la expresión sea una palabra completa.
[[:digit:]]{4} → Encuentra cuatro números consecutivos.
[[:upper:]]{3} → Encuentra tres letras mayúsculas consecutivas.

```
paula@ubuntu:~$ echo "1234ABC" | egrep '\b[[:digit:]]{4}[[:upper:]]{3}\b'
1234ABC
paula@ubuntu:~$ echo "1234567" | egrep '\b[[:digit:]]{4}[[:upper:]]{3}\b'
paula@ubuntu:~$ echo "123456A" | egrep '\b[[:digit:]]{4}[[:upper:]]{3}\b'
paula@ubuntu:~$ echo "9876XYZ" | egrep '\b[[:digit:]]{4}[[:upper:]]{3}\b'
9876XYZ
```

d) Escribir la instrucción para encontrar números de cuatro cifras que no empiecen por 9.

egrep '\b[0-8][0-9]{3}\b' fichero.txt

\b → Marca el inicio y fin de la palabra.
[0-8] → Asegura que el primer dígito no sea 9.
[0-9]{3} → Captura tres dígitos adicionales.

```
paula@ubuntu:~$ echo "0999" | egrep '\b[0-8][0-9]{3}\b'
0999
paula@ubuntu:~$ echo "9786" | egrep '\b[0-8][0-9]{3}\b'
paula@ubuntu:~$ echo "09234" | egrep '\b[0-8][0-9]{3}\b'
0999
paula@ubuntu:~$
```

e) Indicar la instrucción para encontrar los números de cuatro cifras comprendidos entre 1500 y 2000.

egrep '\b1[5-9][0-9]{2}|2000\b' fichero.txt

1[5-9][0-9]{2} → Captura números entre 1500 y 1999.
|2000 → Incluye el número "2000".

```
paula@ubuntu:~$ echo "1499" | egrep '\b1[5-9][0-9]{2}|2000\b'
paula@ubuntu:~$ echo "1512" | egrep '\b1[5-9][0-9]{2}|2000\b'
1512
paula@ubuntu:~$
```

f) Escribir la instrucción para encontrar fechas del siglo XXI del mes de enero en el formato dd-mm-aaaa. Considera la fecha como una palabra.

egrep '\b(0[1-9]|[12][0-9]|3[01])-01-20[0-9]{2}\b' fichero.txt

(0[1-9]|[12][0-9]|3[01]) → Captura días del 01 al 31.
-01- → Indica que el mes es enero.
20[0-9]{2} → Asegura que el año esté entre 2000 y 2099.

```
paula@ubuntu:~$ echo "15-01-2023" | egrep '\b(0[1-9][12][0-9]|3[01])-(0[1-
-9]1[012])-(20\d}})\b'
15-01-2023
paula@ubuntu:~$ echo "13-0325" | egrep '\b(0[1-9]|[12][0-9])-3[01]|11[012)
-(20\d{2})\b'\\\b'
paula@ubuntu:~$
```

g) Obtener la instrucción que permite encontrar números de teléfono de 9 cifras que empiezan por 901.

egrep '\b901[0-9]{6}\b' fichero.txt

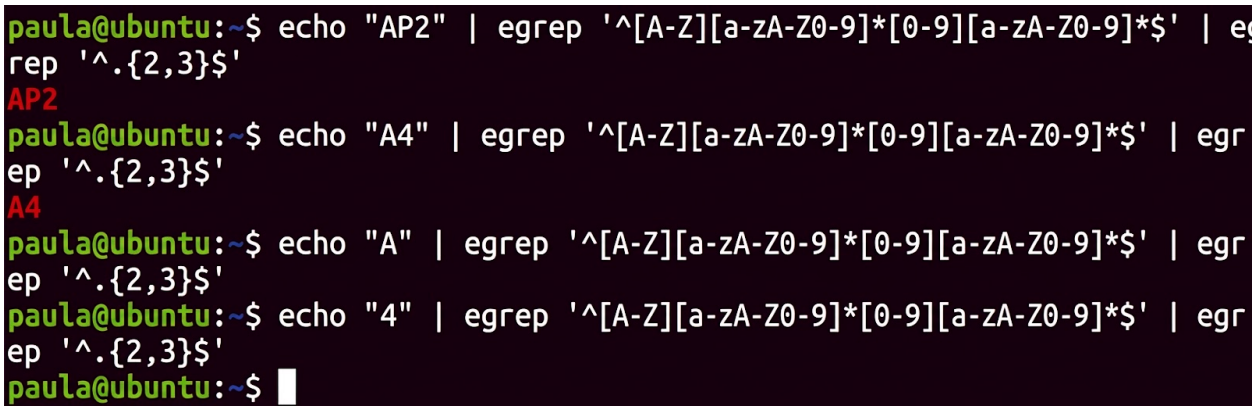
901 → Asegura que el número comience con "901".
[0-9]{6} → Captura los 6 dígitos restantes.

```
paula@ubuntu:~$ echo "901567342"| egrep '\b901[0-9]{6}\b'
901567342
paula@ubuntu:~$ echo "801567342" | egrep '\b901[0-9]{6}\b'
paula@ubuntu:~$
```

Ejercicio 5:

egrep '^[A-Z][a-zA-Z0-9]*[0-9][a-zA-Z0-9]*$' fichero.txt | egrep '^.{2,3}$'

^ → Indica el inicio de la línea.
[A-Z] → Asegura que el primer carácter sea una letra mayúscula.
[a-zA-Z0-9]* → Permite que el resto de caracteres sean alfanuméricos.
[0-9] → Obliga a que haya al menos un número en la contraseña.
[a-zA-Z0-9]*$ → Permite más caracteres alfanuméricos hasta el final de la línea.
^.{2,3}$ → Restringe la longitud de la contraseña a entre 2 y 3 caracteres.

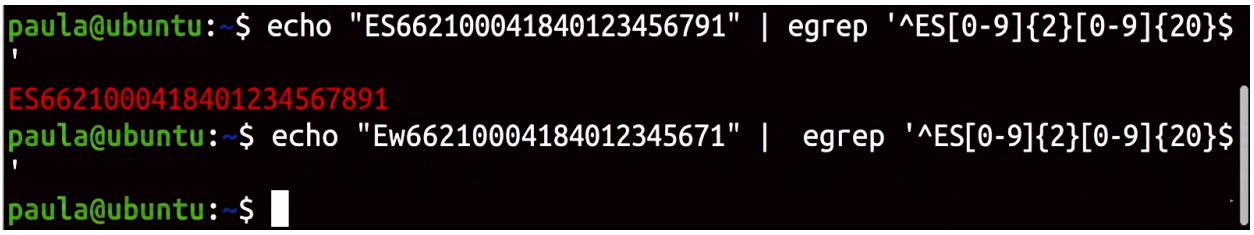


Ejercicio 6:

El IBAN de una cuenta bancaria en España tiene el formato "ESXXYYYYYYYYYYYYYYYYYYYY", donde:
- "ES" son las letras fijas al inicio.
- XX son dos dígitos de control.
- YYYYYYYYYYYYYYYYYYYY es el número de cuenta (20 dígitos).

egrep '^ES[0-9]{2}[0-9]{20}$' fichero.txt

^ES → La cuenta debe empezar con "ES".
[0-9]{2} → Dos dígitos de control.
[0-9]{20} → 20 dígitos del número de cuenta.
$ → Indica que la cadena debe terminar ahí (evita coincidencias parciales).



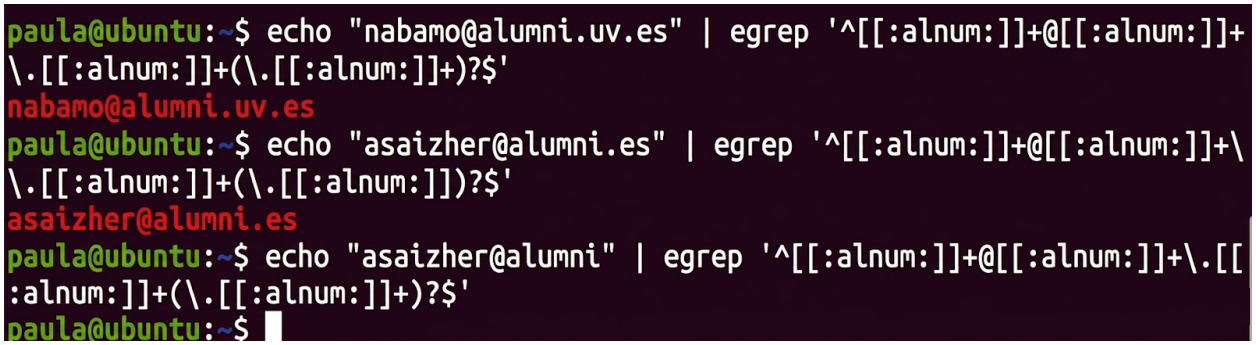
Ejercicio 7:

Una dirección de correo válida tiene el formato: L+ @ L+ . L+ (. L+ opcional), donde L representa caracteres alfanuméricos (a-z, A-Z, 0-9).

egrep '^[[:alnum:]]+@[[:alnum:]]+\.[[:alnum:]]+(\.[[:alnum:]]+)?$' fichero.txt

^ → Inicio de línea.

[[:alnum:]]+ → Nombre de usuario con uno o más caracteres alfanuméricos.
@ → Símbolo obligatorio de "arroba".
[[:alnum:]]+ → Dominio con al menos un carácter alfanumérico.
\. → Punto obligatorio que separa el dominio de la extensión.
[[:alnum:]]+ → Extensión del dominio (por ejemplo, "com", "es", "org").
(\.[[:alnum:]]+)? → Extensión secundaria opcional (.mx, .uk, etc.).
$ → Final de línea.



Ejercicio 8:

egrep '^\<a[[:alpha:]]*a\>|\<a\>?$' fichero.txt

^ → Inicio de la línea.
| → Operador unión que separa las dos opciones de búsqueda (palabras largas o de una sola letra).
< → Indica el inicio de una palabra.
    a → La letra "a" literal al principio de la palabra.
    [[:alpha:]] → Alfabeto predefinido para caracteres alfabéticos (letras).
    * → Cierre de Kleene que indica cero o más repeticiones de las letras en medio de la palabra.
    a → La letra "a" literal que debe ir al final de la palabra.
> → Indica el final de una palabra.
    <a> → Segunda condición del OR: contempla el caso especial de la palabra "a"
$ → Final de la línea.

Ejercicio 9:

egrep '^([[:digit:]]{4}-){3}[[:digit:]]{4}$' fichero.txt

^ → Inicio de la línea.
() → Paréntesis para agrupar la expresión interna y tratarla como un solo bloque.

[[:digit:]] → Alfabeto predefinido que representa cualquier dígito.
{4} → Indica exactamente 4 repeticiones del dígito anterior.
- → El carácter literal del guión actúa como separador.
{3} → Indica que todo el grupo agrupado entre paréntesis (los 4 dígitos seguidos del guión) se repite exactamente 3 veces.
[[:digit:]]{4} → El bloque final de exactamente 4 dígitos, esta vez sin el guión al final.
$ → Final de la línea.

Ejercicio 10:

egrep '^([0-9A-Fa-f]{1,4}:){7}[0-9A-Fa-f]{1,4}$' fichero.txt
^ → Inicio de la línea.
([0-9A-Fa-f]{1,4}:){7} →
[0-9A-Fa-f]{1,4} → Entre 1 y 4 caracteres hexadecimales.
: → Separador.
{7} → Se repite siete veces para los primeros 7 bloques.
[0-9A-Fa-f]{1,4}$ → Último bloque con 1 a 4 caracteres hexadecimales.
$ → Final de la línea.

Ejercicio 11:

egrep '\b(0[1-9]|[1-4][0-9]|5[0-2])[0-9]{3}\b' fichero.txt

Reconoce códigos postales de España, que están formados por 5 dígitos, donde los dos primeros representan una provincia (entre 01 y 52).

Ejercicio 12:

egrep -c '\b(0[1-9]|[1-4][0-9]|5[0-2])[0-9]{3}\b' fichero.txt

La opción “-c (--count)” permite, en este caso, contar cuántos códigos postales válidos aparecen en el fichero de texto.